Mecanismo de Consistência entre os Servidores de Dados

Exclusão mútua distribuída (Ricart-Agrawala) e replicação síncrona

Projeto de Implementação Desktop – Sistemas Distribuídos 2026/2

Dupla: *[preencher os nomes]*

Outubro de 2026

Resumo

Este documento descreve a arquitetura do sistema distribuído desenvolvido no projeto e, em detalhe, o mecanismo de consistência entre os dois servidores de dados. A solução combina exclusão mútua distribuída pelo algoritmo de Ricart-Agrawala, com ordenação dos pedidos por relógios lógicos de Lamport, e replicação síncrona com confirmação (`REPLICAR`/`ACK`). São apresentados os fundamentos teóricos, a justificativa da escolha frente às alternativas, o projeto da implementação, argumentos de correção, o plano de testes e as limitações conhecidas.

Sumário

1 Introdução **2**

2 Visão geral do sistema **2**
- 2.1 Componentes e portas . . . 2
- 2.2 Protocolo de mensagens . . . 2
- 2.3 Decisões de projeto das demais camadas . . . 3

3 Fundamentos teóricos **4**
- 3.1 O problema da exclusão mútua distribuída . . . 4
- 3.2 Relógios lógicos de Lamport . . . 4
- 3.3 O algoritmo de Ricart-Agrawala . . . 5
- 3.4 Comparação com as alternativas . . . 6
- 3.5 Modelos de replicação e consistência . . . 6

4 Mecanismo de consistência proposto **6**
- 4.1 Fluxo de uma requisição . . . 6
- 4.2 Decisões de projeto . . . 7
  - 4.2.1 Duas camadas de exclusão mútua . . . 7
  - 4.2.2 Resposta adiada como *thread* em espera . . . 8
  - 4.2.3 Toda requisição passa pela seção crítica . . . 8
  - 4.2.4 Replicação síncrona com conferência de linha . . . 8
  - 4.2.5 Par indisponível: consistência acima de disponibilidade . . . 8
- 4.3 Estrutura da implementação . . . 9

5 Argumentos de correção **9**

6 Plano de testes **10**

7 Limitações conhecidas **10**

8 Perguntas prováveis na apresentação **11**

1 Introdução

O enunciado do projeto exige um sistema em camadas com 2 clientes, 1 balanceador de carga, 3 servidores de aplicação e 2 servidores de dados, todos comunicando-se por *sockets* TCP. Cada servidor de dados mantém um arquivo local, e o enunciado impõe três requisitos sobre eles:

R1. Após toda escrita (nova linha) em um dos arquivos, a consistência deve ser disparada.

R2. Nenhuma nova requisição pode ser atendida antes do término da consistência.

R3. Ao final de cada escrita e ao término da execução, os dois arquivos devem conter os mesmos valores, sem duplicatas.

Além disso, **nenhuma requisição dos clientes pode ser perdida**: todo número distinto registrado nos logs dos clientes deve estar presente nos arquivos dos servidores de dados.

O mecanismo de consistência vale 3 dos 10 pontos do trabalho e é o foco deste documento (Seções 3 a 7). A Seção 2 apresenta o restante do sistema, necessário para entender o contexto em que a consistência opera.

2 Visão geral do sistema

2.1 Componentes e portas

Todos os componentes são executáveis independentes em `localhost`; portas diferentes simulam máquinas diferentes (Tabela 1).

Tabela 1: Componentes do sistema e portas utilizadas.

Componente	**Porta(s)**	**Papel**
Cliente 1 e 2	–	Sorteia números de 1 a 1000 e envia ao balanceador.
Balanceador	5000	*Proxy* Round Robin entre os servidores de aplicação.
Servidores de Aplicação	6001–6003	Uma *thread worker* por requisição: primalidade e consulta ao servidor de dados.
Servidores de Dados	7001–7002	Atendem os *workers*: verificam e inserem no arquivo local.
Canal de sincronização	7101–7102	Comunicação entre os dois servidores de dados (exclusão mútua e replicação).

2.2 Protocolo de mensagens

O protocolo é textual, com uma mensagem por linha e campos separados por ponto e vírgula (Tabela 2). Mensagens textuais facilitam a depuração (podem ser lidas no console) e a demonstração.

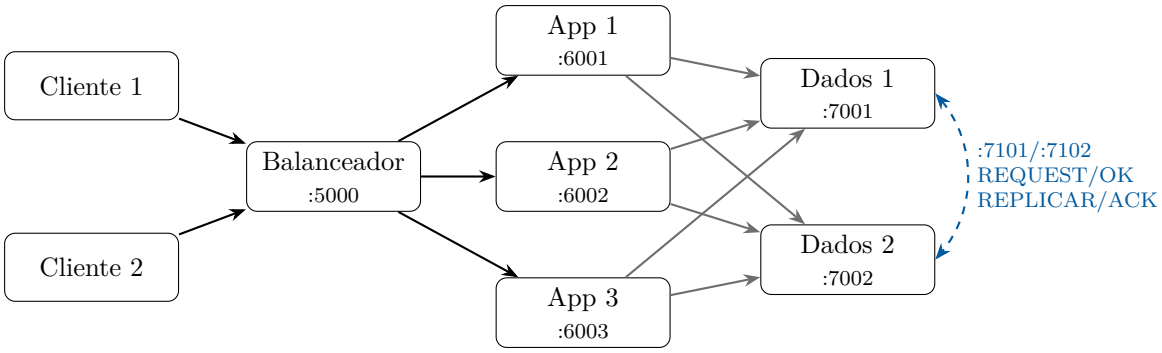


Figura 1: Arquitetura em camadas. Setas cinzas: o *worker* sorteia um dos servidores de dados. Tracejado: canal de sincronização entre os servidores de dados.

Tabela 2: Mensagens trocadas entre os componentes.

Origem → Destino	**Mensagem**	**Significado**
Cliente → Balanc. → App	`CHECK;n`	Pedido de verificação do número n
App → Balanc. → Cliente	`RECEBIDO;n`	Confirmação imediata de recebimento
App → Dados	`CHECK;n`	Consulta/inserção de n
Dados → App	`INSERIDO;n;linha`	n não existia e foi inserido
Dados → App	`EXISTE;n;linha`	n já existia na linha informada
Dados ↔ Dados	`REQUEST;ts;id`	Pedido de entrada na seção crítica
Dados ↔ Dados	`OK;id`	Permissão concedida
Dados ↔ Dados	`REPLICAR;n;linha`	Réplica de uma escrita
Dados ↔ Dados	`ACK`	Réplica gravada

2.3 Decisões de projeto das demais camadas

Balanceador: Round Robin como *proxy*. O cliente conhece um único endereço; o balanceador repassa o `CHECK` ao servidor de aplicação da vez e devolve o `RECEBIDO`. A escolha do Round Robin se justifica porque os servidores são idênticos e as requisições são homogêneas: o teste de primalidade para $n \leq 1000$ custa no máximo cerca de 16 divisões, e o tempo de cada requisição é dominado pela espera no servidor de dados, que é a mesma para qualquer servidor de aplicação. Políticas como *least connections* não teriam informação útil, pois cada conexão entre balanceador e servidor de aplicação dura apenas milissegundos (até o `RECEBIDO`); a carga real fica nos *workers*. O contador é um `AtomicInteger`, atualizado de forma atômica por `getAndUpdate(i -> (i+1) % 3)`, o que evita condições de corrida entre as *threads* do balanceador. Se o servidor da vez estiver indisponível, o balanceador desvia para o próximo da sequência.

Confirmação imediata (`RECEBIDO`). O servidor de aplicação confirma o recebimento antes de processar. Assim o cliente espera apenas milissegundos (e não o processamento completo), mas tem prova de que a requisição chegou ao sistema; sem a confirmação, retransmite.

Servidor de aplicação. Para cada requisição é criada uma nova *thread* (`new Thread`), como exige o enunciado. O *worker* verifica a primalidade primeiro (divisão por 2 e pelos ímpares até $\sqrt{n}$), sorteia um servidor de dados, aguarda a resposta e imprime a frase no formato exato. Se nenhum servidor de dados responder, tenta novamente com espera crescente, sem desistir: a requisição já foi confirmada ao cliente.

Cliente. Grava o número no log *antes* de enviar (com *flush*), e reenvia o *mesmo* número até receber o `RECEBIDO`. O reenvio é seguro porque `CHECK` é **idempotente**: processar o mesmo número duas vezes apenas produz "já se encontra no arquivo" na segunda vez. A parada do cliente acontece sempre entre uma requisição e outra, nunca entre gravar no log e enviar.

Fila de conexões (*backlog*). Durante os testes foi encontrado um defeito real: no Windows, quando a fila de conexões pendentes de um `ServerSocket` (50 por padrão no Java) enche durante uma rajada, o sistema operacional *recusa* as conexões excedentes. Todos os servidores passaram a usar uma fila de 1000 posições.

Servidor de dados. Mantém um índice em memória (número → linha) reconstruído a partir do arquivo na inicialização, o que torna a verificação de existência $O(1)$. A espera aleatória de 100 a 300 ms ocorre *fora* da seção crítica: as esperas de várias requisições se sobrepõem, e apenas a parte que exige exclusividade (verificar, escrever e replicar) é serializada. Com a espera dentro da seção crítica, a vazão máxima seria de cerca de $1/0{,}2\,\mathrm{s} = 5$ requisições por segundo para o sistema inteiro, contra cerca de 16 geradas pelos dois clientes, e a fila cresceria sem limite.

3 Fundamentos teóricos

3.1 O problema da exclusão mútua distribuída

Uma **seção crítica** (SC) é um trecho de código que acessa um recurso compartilhado e não pode ser executado por mais de um processo ao mesmo tempo. Em um único processo, basta um *lock*. Em um sistema distribuído não há memória compartilhada nem relógio global, e os processos só se coordenam por mensagens. Um algoritmo de exclusão mútua distribuída deve garantir [3]:

Segurança (*safety*) no máximo um processo executa a SC a cada instante;

Vivacidade (*liveness*) todo pedido de entrada é atendido em algum momento, o que exclui *deadlock* e inanição (*starvation*);

Ordenação se um pedido aconteceu antes de outro (no sentido de Lamport), a entrada na SC respeita essa ordem.

No projeto, a SC é *verificar se o número existe, inserir se necessário e replicar no outro servidor*. Se dois servidores verificassem ao mesmo tempo, ambos poderiam concluir que o número não existe e inseri-lo, gerando duplicata ou linhas divergentes.

3.2 Relógios lógicos de Lamport

Lamport [1] define a relação **aconteceu-antes** ($\rightarrow$): $a \rightarrow b$ se a e b ocorrem no mesmo processo e a vem antes, ou se a é o envio de uma mensagem e b é o seu recebimento (e por transitividade). Eventos não relacionados são **concorrentes**.

Cada processo P_i mantém um contador C_i, atualizado pelas regras:

LC1. Antes de cada evento local (incluindo o envio), $C_i \leftarrow C_i + 1$; uma mensagem enviada carrega o valor $t = C_i$.

LC2. Ao receber uma mensagem com marca t, $C_i \leftarrow \max(C_i, t) + 1$.

Essas regras garantem que $a \to b \Rightarrow C(a) < C(b)$. A recíproca não vale: marcas menores não implicam causalidade. Para obter uma **ordem total**, desempata-se pelo identificador do processo:

$$(t_i, i) < (t_j, j) \iff t_i < t_j \ \vee \ (t_i = t_j \wedge i < j).$$

É essa ordem total que decide quem entra primeiro na SC.

3.3 O algoritmo de Ricart-Agrawala

Ricart e Agrawala [2] propuseram um algoritmo totalmente distribuído, baseado em multicast de pedidos e respostas adiadas. Cada processo tem um estado $\in \{\textsc{Livre}, \textsc{Querendo}, \textsc{NaSC}\}$ (Algoritmo 1).

Algoritmo 1 Ricart-Agrawala para o processo P_i com N processos.

```
1:  procedimento ENTRAR
2:      estado ← QUERENDO;  C_i ← C_i + 1;  T_i ← C_i
3:      enviar REQUEST(T_i, i) aos outros N − 1 processos
4:      aguardar N − 1 respostas OK
5:      estado ← NaSC
6:  fim procedimento

7:  procedimento AoReceberRequest(T_j, j)
8:      C_i ← max(C_i, T_j) + 1
9:      se estado = NaSC ou (estado = QUERENDO e (T_i, i) < (T_j, j)) então
10:         adiar a resposta (enfileirar j)
11:     senão
12:         enviar OK a P_j
13:     fim se
14: fim procedimento

15: procedimento SAIR
16:     estado ← LIVRE
17:     enviar OK a todos os pedidos adiados
18: fim procedimento
```

Custo. Cada entrada na SC custa $2(N-1)$ mensagens: $N-1$ pedidos e $N-1$ respostas. Com $N = 2$, são **duas mensagens** (`REQUEST` e `OK`). O atraso de sincronização (tempo entre um processo sair e o próximo entrar) é de uma única mensagem, o `OK` adiado.

3.4 Comparação com as alternativas

Tabela 3: Algoritmos clássicos de exclusão mútua distribuída, com N processos.

Algoritmo	**Msgs/entrada**	**Atraso de sinc.**	**Observações para o projeto**
Coordenador central	3	2 msgs	Exige um processo extra ou tornar um servidor "chefe" (assimetria); ponto único de falha.
Token ring	1 a ∞	0 a $N-1$ msgs	O token circula mesmo sem requisições; com 2 nós ele fica "quicando" continuamente; perda do token exige recuperação.
Ricart-Agrawala	$2(N-1)$	1 msg	Simétrico, sem coordenador; nenhum tráfego quando ocioso; com $N=2$, apenas `REQUEST` e `OK`.

A fraqueza conhecida do Ricart-Agrawala é que a falha de *qualquer* processo bloqueia os demais, pois todos precisam responder. Com apenas dois servidores de dados e replicação síncrona, essa propriedade não é uma desvantagem adicional: se o par está fora do ar, a replicação também não pode ser concluída, e o sistema precisa esperar de qualquer forma (Seção 4.2.5).

3.5 Modelos de replicação e consistência

Replicar é manter cópias de um mesmo dado em nós diferentes. As escolhas principais são [4]:

- **Síncrona ou assíncrona**: na síncrona, a escrita só é considerada concluída quando todas as réplicas a confirmam; na assíncrona, as réplicas são atualizadas depois, o que permite janelas de divergência.
- **Primário fixo ou escrita em qualquer réplica**: no primeiro, todas as escritas passam por um nó; no segundo, qualquer réplica aceita escritas e é preciso coordená-las.

A solução adotada é **escrita em qualquer réplica, coordenada por exclusão mútua distribuída, com replicação síncrona**. Como as escritas acontecem uma de cada vez (a SC é global) e cada uma só termina após o `ACK` da réplica, todas as escritas formam uma única sequência observada igualmente pelos dois servidores. Isso caracteriza **consistência forte**: qualquer leitura, em qualquer servidor, reflete todas as escritas concluídas antes dela.

4 Mecanismo de consistência proposto

4.1 Fluxo de uma requisição

A Figura 2 mostra uma inserção iniciada no servidor de dados 1, enquanto o servidor 2 também deseja entrar na SC.

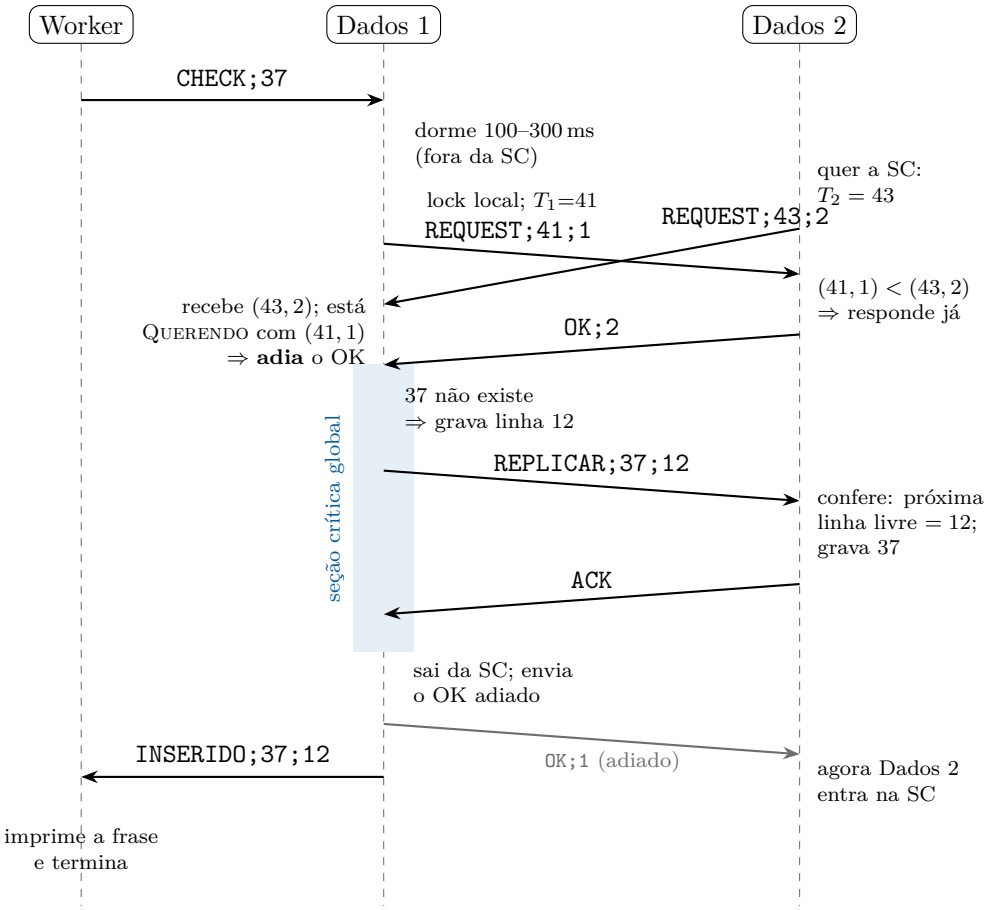


Figura 2: Diagrama de sequência de uma inserção com pedidos concorrentes. Os dois servidores pedem a SC quase ao mesmo tempo (as setas se cruzam); como $(41,1) < (43,2)$, Dados 1 entra primeiro e Dados 2 recebe o `OK` adiado quando Dados 1 sai.

Em texto, o servidor que recebe `CHECK;n`:

1. dorme de 100 a 300 ms (fora da SC);
2. chama `entrar()`: obtém o *lock* local, incrementa o relógio, envia `REQUEST` ao par e aguarda o `OK`;
3. na SC: verifica se n existe; se existir, prepara `EXISTE`; se não, grava n na próxima linha, envia `REPLICAR;n;linha` e aguarda o `ACK`;
4. chama `sair()`: libera os pedidos adiados e o *lock* local;
5. responde ao *worker*.

4.2 Decisões de projeto

4.2.1 Duas camadas de exclusão mútua

O Ricart-Agrawala coordena *processos*; porém cada servidor de dados atende várias *threads* de *worker* simultaneamente. Por isso `entrar()` tem duas camadas:

1. um ***lock* local justo** (`ReentrantLock(true)`, ordem de chegada) escolhe qual *thread* do servidor vai disputar a SC;
2. essa *thread* executa o Ricart-Agrawala com o par.

A alternância justa entre os servidores decorre do próprio algoritmo: quando o servidor 1 sai da SC, sua próxima *thread* pede com uma marca nova, maior que a do pedido pendente do servidor 2 (pois o relógio do servidor 1 foi atualizado ao receber esse pedido, regra LC2). Logo o servidor 2 tem prioridade e entra em seguida; nenhum servidor monopoliza a SC.

4.2.2 Resposta adiada como *thread* em espera

Cada mensagem usa uma conexão TCP própria, no modelo pergunta–resposta, seguindo o padrão do restante do sistema. Quando um `REQUEST` chega, a *thread* que atende a conexão avalia a regra do Algoritmo 1; se precisar adiar, ela espera (`wait()`) até que `sair()` a acorde (`notifyAll()`), e então escreve `OK` na mesma conexão. Isso implementa fielmente o "adiar a resposta" do algoritmo, sem necessidade de manter conexões persistentes, tratar reconexões ou garantir ordem FIFO entre mensagens distintas.

4.2.3 Toda requisição passa pela seção crítica

Foi considerado um atalho: se o número já está no índice local, responder `EXISTE` sem pedir permissão ao par. Seria mais rápido, pois com o tempo a maioria das requisições vira `EXISTE`. O atalho foi **rejeitado** porque violaria o requisito R2: um servidor poderia atender uma consulta enquanto recebe um `REPLICAR`, isto é, durante a consistência. O custo de seguir o enunciado é pequeno: a SC dura poucos milissegundos em `localhost`, ante cerca de 16 requisições por segundo.

4.2.4 Replicação síncrona com conferência de linha

`REPLICAR;n;linha` carrega a linha em que n foi gravado. O servidor que recebe confere que a próxima linha livre do seu arquivo é exatamente essa e que n não existe; qualquer divergência é registrada como erro. Como a SC é global, essa verificação nunca deve falhar, e por isso funciona como evidência, durante a execução, de que os arquivos permanecem idênticos.

O tratamento de `REPLICAR` *não* pede a SC: quem o envia já a detém, e o servidor que recebe está, por construção, impedido de entrar nela (seu pedido aguarda o `OK`). A escrita local continua protegida pelo *lock* interno do arquivo (métodos `synchronized`), pois chega por outra *thread*.

4.2.5 Par indisponível: consistência acima de disponibilidade

Se o par não responde, o pedido ou a réplica são reenviados com espera crescente (50 ms dobrando até 1 s) até o par voltar. O servidor *não* prossegue sozinho.

Pelo teorema CAP [5, 6], um sistema distribuído sujeito a partições de rede não pode garantir simultaneamente consistência e disponibilidade. O projeto escolhe **consistência**: prosseguir sozinho deixaria os arquivos divergentes, contrariando R3. Nenhuma requisição é perdida, pois todas ficam aguardando e são atendidas quando o par volta. A mesma regra resolve a ordem de inicialização: um servidor que sobe antes do outro simplesmente espera.

4.3 Estrutura da implementação

Tabela 4: Classes envolvidas no mecanismo de consistência.

Classe	Responsabilidade
`ExclusaoMutua`	Interface com `entrar()` e `sair()`. O servidor só conhece esta abstração, o que permite trocar o algoritmo sem alterar o servidor.
`RicartAgrawala`	Implementa a interface: estado, relógio de Lamport, *lock* local, pedidos e respostas adiadas.
`CanalSync`	`ServerSocket` na porta de sincronização; trata `REQUEST` e `REPLICAR`; envia mensagens ao par com reenvio.
`ArquivoDados`	Arquivo e índice em memória; `consultarOuInserir(n)` e `inserirReplicado(n, linha)`.
`ServidorDados`	Atende os *workers* e orquestra: dormir, `entrar()`, verificar, gravar e replicar, `sair()`.

O núcleo do servidor de dados fica assim:

```
Thread.sleep(100 a 300 ms); // fora da seção crítica
exclusao.entrar(); // lock local + Ricart-Agrawala
try {
    r = arquivo.consultarOuInserir(n); // verificação DENTRO da SC
    if (r.inserido)
        canal.replicar(n, r.linha); // REPLICAR -> aguarda ACK
} finally {
    exclusao.sair(); // libera pedidos adiados
}
responder(r); // INSERIDO ou EXISTE
```

5 Argumentos de correção

Segurança (no máximo um servidor na SC). Suponha, por absurdo, que os dois servidores estejam na SC ao mesmo tempo. Cada um recebeu o `OK` do outro. Há dois casos:

- *Os pedidos foram concorrentes*: ambos estavam QUERENDO ao receber o pedido do outro. Como $(T_1, 1)$ e $(T_2, 2)$ são comparáveis na ordem total, exatamente um deles é menor; o servidor de pedido maior responde e o de pedido menor adia. Logo apenas um recebeu `OK`.
- *Um respondeu enquanto estava* LIVRE: digamos que o servidor 2 respondeu ao pedido $(T_1, 1)$. Pela regra LC2, $C_2 > T_1$ a partir daí, então um pedido posterior do servidor 2 terá $T_2 > T_1$. O servidor 1, QUERENDO ou NASC, adiará esse pedido até sair.

Em ambos os casos há contradição. Dentro de cada servidor, o *lock* local garante uma única *thread* disputando; portanto no máximo uma *thread* do sistema inteiro está na SC.

Ausência de *deadlock*. Um *deadlock* exigiria que cada servidor adiasse o pedido do outro. Isso só ocorre se cada pedido for menor que o outro na ordem total, o que é impossível.

Ausência de inanição. Os pedidos são atendidos em ordem crescente de (T, id). Um pedido pendente tem marca fixa, e todo pedido novo do outro servidor tem marca maior (regra LC2), então o pedido pendente é atendido no máximo após a SC em curso. Localmente, o *lock* justo atende as *threads* na ordem de chegada.

Arquivos idênticos e sem duplicatas. Toda escrita ocorre na SC global, após verificação no índice, e só termina após o `ACK` da réplica. As escritas são, portanto, totalmente ordenadas e aplicadas na mesma ordem e nas mesmas linhas nos dois arquivos; a conferência de linha no `REPLICAR` verifica isso a cada escrita.

Nenhuma requisição perdida. A garantia é de ponta a ponta, em cadeia:

1. o cliente grava no log antes de enviar e reenvia até receber `RECEBIDO`;
2. o balanceador desvia para outro servidor de aplicação em caso de falha;
3. o *worker*, após confirmar, nunca desiste do servidor de dados;
4. o servidor de dados bloqueia (não descarta) requisições durante a consistência e espera o par se ele estiver indisponível;
5. os reenvios são seguros porque `CHECK` é idempotente;
6. a parada do sistema encerra primeiro os clientes (entre requisições) e espera os servidores esvaziarem antes de fechá-los.

6 Plano de testes

1. **Execução completa** (cerca de 30 s, depois parada ordenada): `dados1.txt` e `dados2.txt` idênticos; nenhum com duplicatas; todo número distinto dos logs presente nos arquivos; número de frases impressas igual ao número de linhas dos logs.
2. **Estresse da exclusão mútua**: 100 requisições simultâneas ao servidor 1 e outras 100 ao servidor 2, ao mesmo tempo, com números de 1 a 30. Nenhum número pode ser inserido duas vezes no conjunto dos dois servidores, e cada linha deve conter um único número.
3. **Par indisponível**: derrubar o servidor 2 com o sistema em execução. As requisições do servidor 1 devem aguardar, sem perda, e ser retomadas quando o servidor 2 voltar; os arquivos devem permanecer idênticos.
4. **Unicidade das linhas**: nas frases dos três servidores de aplicação, nenhuma linha pode aparecer como "inserido na linha Y" mais de uma vez.

7 Limitações conhecidas

- **Falha dentro da SC**: se um servidor morrer depois de gravar localmente e antes de o `REPLICAR` chegar ao par, os arquivos ficam com uma linha de diferença. Resolver isso exigiria um protocolo de recuperação (sincronizar o estado ao reconectar) ou de *commit* atômico (por exemplo, *two-phase commit*), o que extrapola o escopo do enunciado.
- **Disponibilidade**: por escolha de projeto (Seção 4.2.5), se um servidor de dados cair, o outro deixa de concluir requisições até ele voltar. As requisições não se perdem, mas aguardam.
- **Escalabilidade**: com N servidores, cada entrada custaria $2(N-1)$ mensagens e todos precisariam estar ativos. Para dois servidores o custo é mínimo.
- **Vazão**: toda requisição, inclusive de números já existentes, passa pela SC global. A vazão é limitada pela duração da SC (poucos milissegundos), suficiente para a carga do projeto.

8 Perguntas prováveis na apresentação

Por que não usar um *lock* comum, como `synchronized`?
Ele vale apenas dentro de um processo. Os dois servidores de dados são processos independentes (simulando máquinas diferentes), sem memória compartilhada; a coordenação precisa ser por mensagens.

Para que serve o relógio de Lamport, se só há dois processos?
Para decidir, sem relógio físico comum, qual de dois pedidos simultâneos tem prioridade, e para garantir que um pedido feito depois de receber outro tenha marca maior, o que evita inanição.

O que acontece se os dois pedirem ao mesmo tempo com a mesma marca?
Desempata-se pelo identificador: $(41, 1) < (41, 2)$, então o servidor 1 entra primeiro e o servidor 2 entra assim que ele sai.

Por que a verificação de existência fica dentro da SC?
Fora dela, os dois servidores poderiam verificar o mesmo número ao mesmo tempo, ambos concluir que ele não existe e ambos inseri-lo.

Por que replicação síncrona e não assíncrona?
O enunciado exige que nenhuma requisição seja atendida antes do fim da consistência e que os arquivos sejam iguais a cada escrita. A replicação assíncrona abriria uma janela em que os arquivos divergem.

Por que não token ring?
Com dois nós, o token circularia continuamente mesmo sem requisições, e a perda do token exigiria um mecanismo de recuperação. O Ricart-Agrawala só troca mensagens quando alguém quer entrar na SC.

E se um servidor de dados cair?
O outro espera e reenvia; as requisições ficam retidas, não perdidas. Foi escolhida a consistência acima da disponibilidade (teorema CAP).

Referências

[1] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[2] G. Ricart e A. K. Agrawala. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, 24(1):9–17, 1981.

[3] G. Coulouris, J. Dollimore, T. Kindberg e G. Blair. *Distributed Systems: Concepts and Design*. 5. ed. Addison-Wesley, 2011.

[4] A. S. Tanenbaum e M. van Steen. *Distributed Systems*. 3. ed. Pearson, 2017.

[5] E. A. Brewer. Towards robust distributed systems. Palestra no *ACM Symposium on Principles of Distributed Computing (PODC)*, 2000.

[6] S. Gilbert e N. Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.